UNIVERSIDAD EAFIT - SISTEMAS OPERATIVOS



Informe de sustentación del editor Unix

Comandos base, memoria dinámica y acceso POSIX de bajo nivel

5 comandos obligatorios	6 interfaces POSIX clave	24 funciones propias

Javier Andrés Sierra Machado | Entrega individual | SO2026B

1. Alcance y cumplimiento

La entrega implementa el nivel individual como ejecutable independiente.

El proyecto desarrolla un editor interactivo de archivos planos en C. El acceso al documento no usa fopen, fread, fwrite ni fclose: se realiza mediante interfaces POSIX de bajo nivel. El documento se representa temporalmente como una lista doblemente enlazada para practicar memoria dinámica y punteros.

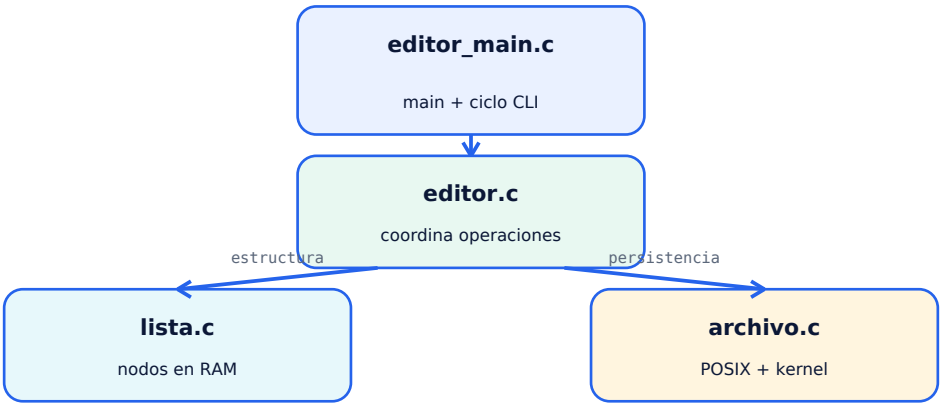
Requisito	Implementación	Evidencia
`o archivo`	Abre o crea y carga líneas en RAM.	open, lseek y read.
`p [n]`	Imprime todo o una línea.	Recorrido de nodos y write a FD 1.
`a texto`	Añade una línea al final.	malloc, lseek(SEEK_END) y write.
`d n`	Elimina una línea y guarda.	free, lseek, ftruncate y write.
`q`	Cierra y libera los recursos.	close y free de todos los nodos.

Resultado verificable
make genera el ejecutable editor sin advertencias. make test valida comandos base, archivos sin LF final, entradas inválidas, apertura por argumento y la restricción de I/O.

Alcance consciente
No se implementan i, s, m, y, x, undo/redo, múltiples buffers ni locks porque corresponden a equipos de dos a cinco integrantes.

2. Decisión arquitectónica

Un ejecutable independiente con módulos pequeños y responsabilidades claras.



Módulo	Responsabilidad	Razón
editor_main.c	Banner, entrada y análisis de comandos.	Mantiene el REPL separado de la lógica.
editor.c	Coordina lista y persistencia.	Define las operaciones del dominio.
lista.c	Administra nodos y memoria dinámica.	Aísla punteros, enlaces y liberación.
archivo.c	Accede al documento mediante POSIX.	Concentra fd, offset, buffers y errores.

Decisión acordada con el docente
Por indicación directa del profesor, esta entrega no modifica ni incluye el Shell SO2026B. El editor se compila y ejecuta por sí solo con make y ./editor.

Pensamiento crítico
La independencia reduce acoplamiento y permite probar el editor sin depender del estado o de modificaciones locales del Shell. La modularidad interna conserva una base escalable.

3. Panorama general

Primero entiende dónde vive cada cosa; después estudia las funciones.

El editor mantiene dos representaciones del documento: una persistente en el sistema de archivos y otra temporal como lista enlazada en la memoria RAM.



Zona	Qué contiene	Quién la administra
Disco / sistema de archivos	Los bytes persistentes de notas.txt.	Kernel y sistema de archivos.
Kernel	Tabla de descriptores, offset y objeto de archivo abierto.	Sistema operativo.
RAM del proceso	Editor, buffers, nodos y textos.	Programa y biblioteca de C.
Terminal	Logo, prompt, mensajes y contenido impreso.	Driver de terminal y kernel.

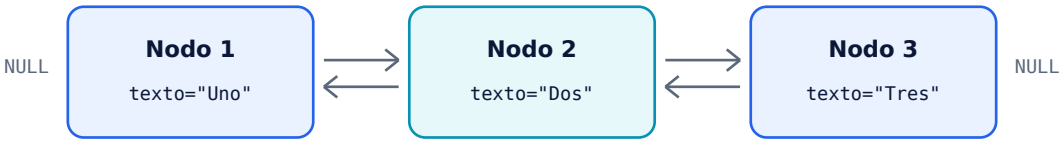
Idea central
El fd no es el archivo ni contiene sus bytes. Es un entero que identifica una entrada de la tabla de descriptores del proceso.

4. Lista doblemente enlazada

Cada línea del documento es un nodo independiente en el heap.

```
typedef struct NodoLinea {
    char *texto;
    size_t longitud;
    struct NodoLinea *anterior;
    struct NodoLinea *siguiente;
} NodoLinea;

typedef struct {
    NodoLinea *primera;
    NodoLinea *ultima;
    size_t cantidad;
} ListaLineas;
```



Cada nodo conoce al anterior y al siguiente

Operación	Trabajo	Costo
Agregar al final	Usa directamente lista->ultima.	O(1)
Buscar la línea n	Recorre desde el extremo más cercano.	O(n)
Borrar un nodo ya localizado	Reconecta anterior y siguiente.	O(1)
Vaciar el documento	Recorre y libera cada nodo.	O(n)

Por qué es doble
Un nodo posee anterior y siguiente. Esto permite recorrer en ambos sentidos y actualizar correctamente los vecinos al borrar.

5. Qué hace cada comando

La interfaz sigue siendo simple aunque por debajo use memoria dinámica y llamadas al sistema.

Comando	RAM	Kernel / archivo
`o notas.txt`	Construye una lista nueva con un nodo por línea.	open, lseek y read.
`p`	Recorre todos los nodos.	write hacia stdout, fd 1.
`p 2`	Localiza el nodo 2 desde el extremo más cercano.	write hacia stdout.
`a texto`	malloc, copia y enlaza un nodo al final.	lseek(SEEK_END), lectura del último byte y write.
`d 2`	Desenlaza el nodo 2 y ejecuta free.	lseek, ftruncate y write.
`e`	Recorre nodos y muestra sus direcciones.	La salida de printf termina en la terminal.
`q`	Libera todos los nodos.	close libera el descriptor.

Dos estrategias de persistencia

```
a texto: lseek(fd, 0, SEEK_END) -> write(texto) -> write("\n")
d n:     lseek(fd, 0, SEEK_SET) -> ftruncate(fd, 0)
         -> write de todas las líneas restantes
```

Consecuencia
Las modificaciones se guardan inmediatamente. a añade al final; d reescribe porque al borrar deben desplazarse todos los bytes posteriores.

6. Inventario del editor: llamadas al sistema

Son las seis interfaces POSIX usadas directamente para manipular el documento.

Llamada	Archivo	Propósito	Resultado importante
`open()`	archivo.c	Abrir o crear el archivo.	fd >= 0; -1 si falla.
`read()`	archivo.c	Copiar bytes hacia RAM.	> 0 bytes; 0 EOF; -1 error.
`write()`	archivo.c / editor.c	Enviar bytes al archivo o stdout.	>= 0 bytes; puede ser parcial.
`lseek()`	archivo.c	Mover el offset.	Nuevo offset; -1 error.
`ftruncate()`	archivo.c	Cambiar el tamaño del archivo.	0 éxito; -1 error.
`close()`	archivo.c	Liberar el descriptor.	0 éxito; -1 error.

Precisión rigurosa: el código llama funciones POSIX expuestas por la biblioteca del sistema. Sus wrappers preparan los registros y ejecutan la transición de modo usuario a modo kernel. En clase normalmente las llamamos syscalls.

Clasificación para la sustentación

Elemento	Clasificación rigurosa	Papel
`open/read/write`	Interfaces POSIX que llegan al kernel.	Mueven datos entre proceso y archivo.
`lseek/ftruncate/close`	Interfaces POSIX que llegan al kernel.	Controlan offset, tamaño y descriptor.
`printf/fgets`	Funciones de la biblioteca estándar de C.	Solo interfaz con la terminal.
`malloc/free`	Funciones del asignador de la biblioteca.	Administran memoria dinámica del proceso.

Ruta conceptual

```
programa C -> wrapper POSIX -> instrucción syscall -> kernel -> driver/VFS
```

7. Syscalls en detalle I

Abrir, leer y posicionar.

open()	`int open(const char *ruta, int flags, mode_t modo);`
Dónde	archivo.c
Qué hace	Abre un objeto del sistema de archivos y crea una entrada en la tabla de descriptores.
Retorno	Devuelve un fd no negativo. Devuelve -1 y establece errno si falla.
En este editor	Se usa como open(nombre, O_RDWR | O_CREAT, 0644). O_RDWR permite leer y escribir; O_CREAT crea si falta. El umask puede retirar permisos del 0644 solicitado.

read()	`ssize_t read(int fd, void *buffer, size_t cantidad);`
Dónde	archivo.c
Qué hace	Copia hasta cantidad bytes desde el archivo abierto hacia memoria del proceso.
Retorno	Positivo: bytes leídos. Cero: EOF. -1: error.
En este editor	Lee bloques de 4096 bytes. El programa examina cada byte y crea un nodo al encontrar LF ('\n').

lseek()	`off_t lseek(int fd, off_t desplazamiento, int origen);`
Dónde	archivo.c
Qué hace	Cambia el offset asociado con la descripción de archivo abierta.
Retorno	Devuelve el nuevo offset; -1 indica error.
En este editor	SEEK_SET regresa al byte cero para cargar o reescribir. SEEK_END ubica el append al final. lseek no lee, no escribe ni cambia por sí sola el tamaño.

Offset
Es la posición desde la cual ocurrirá el próximo read o write sobre esa descripción de archivo.

8. Syscalls en detalle II

Cambiar tamaño, escribir y cerrar.

ftruncate()	`int ftruncate(int fd, off_t longitud);`
Dónde	archivo.c
Qué hace	Ajusta el tamaño del archivo abierto.
Retorno	Cero si tiene éxito; -1 si falla.
En este editor	Después de d, ftruncate(fd, 0) elimina el contenido anterior antes de reescribir la lista. No cambia el offset por sí sola.

write()	`ssize_t write(int fd, const void *buffer, size_t cantidad);`
Dónde	archivo.c y editor.c
Qué hace	Copia bytes desde memoria hacia el objeto indicado por el descriptor.
Retorno	Devuelve cuántos bytes escribió; puede ser menor que cantidad. -1 indica error.
En este editor	a escribe al final; d reescribe la lista. Hacia STDOUT_FILENO (1), p muestra líneas. escribir_todo() repite write hasta completar.

close()	`int close(int fd);`
Dónde	archivo.c
Qué hace	Retira el descriptor de la tabla del proceso y reduce la referencia al archivo abierto.
Retorno	Cero si tiene éxito; -1 si falla.
En este editor	Se llama al cambiar de archivo y al salir. Después no se debe reutilizar ese número como si aún representara el mismo archivo.

Lo que el proyecto no usa
No llama fsync(). close() termina el uso del descriptor, pero no ofrece por sí solo una garantía absoluta frente a un corte eléctrico inmediato.

9. Biblioteca de C: entrada, salida y errores

Seis funciones de <stdio.h> usadas por el código añadido.

Función	Contrato esencial	Uso exacto
`printf()`	Formatea texto y lo coloca en stdout con buffering de biblioteca.	Logo, prompt, mensajes y comando e.
`fprintf()`	Formatea texto hacia un stream seleccionado.	Errores de uso enviados a stderr.
`fgets()`	Lee hasta size-1 caracteres; conserva el salto de línea si cabe.	Lee una orden completa desde stdin.
`fflush()`	Vacía el buffer de salida del stream.	Hace visible editor> antes de bloquearse esperando entrada.
`feof()`	Consulta si el indicador EOF del stream ya está activo.	Ayuda a diferenciar fin de entrada de una orden demasiado larga.
`perror()`	Imprime un prefijo y el texto asociado al errno actual.	Explica fallos al abrir o cerrar.

Biblioteca no significa kernel

printf y fgets son funciones de biblioteca. Internamente pueden terminar usando write o read, pero en main.c nosotros no invocamos esas syscalls directamente.

Trampa común con feof

feof no predice el final. Informa que el indicador EOF quedó activo después de que una operación de lectura encontró el final.

10. Biblioteca de C: memoria y conversión

Cuatro funciones de <stdlib.h>.

Función	Qué hace	Detalle importante en el editor
`malloc()`	Reserva un bloque del heap.	Se pide memoria para NodoLinea y para su texto. NULL significa fallo.
`realloc()`	Cambia el tamaño de un bloque existente.	Duplica el buffer temporal durante la carga; puede moverlo a otra dirección.
`free()`	Devuelve un bloque al asignador.	Libera buffer temporal, texto y nodo. free(NULL) es válido.
`strtol()`	Convierte caracteres a long según una base.	Convierte n en p n y d n usando base 10.

```
NodoLinea *nuevo = malloc(sizeof(*nuevo));
nuevo->texto = malloc(longitud + 1);
...
free(objetivo->texto);
free(objetivo);
```

¿malloc es syscall?
No. malloc, realloc y free pertenecen al asignador de la biblioteca. Este puede solicitar regiones al kernel con brk o mmap, pero esas llamadas no aparecen directamente en nuestro código.

Validación de strtol
convertir_numero_linea verifica ERANGE, que exista al menos un dígito, que no sobren caracteres y que la numeración empiece en 1.

11. Biblioteca de C: cadenas y bytes

Seis funciones de <string.h>.

Función	Resultado	Uso exacto
`strcmp()`	0 si las cadenas completas son iguales.	Reconoce p, a, e y q.
`strncmp()`	Compara como máximo n bytes.	Reconoce los prefijos o , p , a y d .
`strchr()`	Puntero a la primera coincidencia o NULL.	Busca LF para saber si la orden cupo en comando[2048].
`strcspn()`	Cantidad de bytes antes de cualquier carácter rechazado.	Localiza CR o LF y allí escribe '\0'.
`strlen()`	Longitud antes del terminador '\0'.	Calcula los bytes del texto agregado.
`memcpy()`	Copia exactamente n bytes; devuelve destino.	Copia texto al nodo. No añade '\0'; el código lo hace manualmente.

Ejemplo del retiro del salto de línea

```
comando[strcspn(comando, "\r\n")] = '\0';
```

Cadena válida en C
El carácter nulo '\0' marca el final lógico. LF ('\n') es un salto de línea; no son lo mismo.

12. Funciones propias I

Estas no pertenecen al kernel ni a la biblioteca: fueron creadas para el proyecto.

Archivo	Función	Responsabilidad
editor_main.c	`mostrar_bienvenida()`	Imprime el logo Editor Shell.
editor_main.c	`mostrar_resultado()`	Traduce códigos internos a mensajes.
editor_main.c	`convertir_numero_linea()`	Valida y convierte n para p y d.
editor_main.c	`main()`	Inicializa, interpreta comandos y controla el ciclo.
editor.c	`escribir_todo() [static]`	Completa escrituras parciales hacia stdout.
editor.c	`editor_inicializar()`	Coloca fd=-1 e inicializa la lista.
editor.c	`editor_abrir()`	Carga una lista temporal y reemplaza el documento activo.
editor.c	`editor_imprimir()`	Recorre e imprime todos los nodos o uno.
editor.c	`editor_agregar()`	Añade un nodo y sincroniza el archivo.
editor.c	`editor_borrar()`	Borra un nodo y sincroniza el archivo.
editor.c	`editor_mostrar_estructura()`	Expone direcciones y enlaces para estudiar.
editor.c	`editor_cerrar()`	Cierra fd y libera la lista.

Qué significa static aquí
Una función static a nivel de archivo solo es visible dentro de esa unidad de compilación. Por eso editor.c y archivo.c pueden tener cada uno su propia escribir_todo().

13. Funciones propias II

Módulos de estructura de datos y persistencia.

Archivo	Función	Responsabilidad
lista.c	`lista_inicializar()`	Coloca primera y ultima en NULL; cantidad en 0.
lista.c	`lista_vaciar()`	Libera todos los textos y nodos.
lista.c	`lista_agregar()`	Reserva, copia y enlaza un nodo al final.
lista.c	`lista_obtener()`	Busca desde el extremo más cercano.
lista.c	`lista_borrar()`	Reconecta vecinos y libera el nodo objetivo.
archivo.c	`escribir_todo() [static]`	Completa escrituras parciales hacia el archivo.
archivo.c	`agregar_byte() [static]`	Amplía el buffer y añade un byte durante la carga.
archivo.c	`archivo_abrir()`	Envuelve open con los flags del proyecto.
archivo.c	`archivo_cerrar()`	Envuelve close.
archivo.c	`archivo_cargar()`	Convierte bytes del archivo en nodos.
archivo.c	`archivo_agregar_linea()`	Añade con SEEK_END y permite restaurar el tamaño.
archivo.c	`archivo_guardar()`	Reescribe el archivo recorriendo la lista.

Total propio

El editor define 24 funciones: 4 de entrada, 8 de coordinación, 5 de lista y 7 de archivo, contando los auxiliares static.

14. Memoria, fd, buffer y offset

Cuatro conceptos que suelen confundirse.

Concepto	Qué es	Ejemplo del proyecto
Stack	Memoria automática de cada función.	comando[2048], bloque[4096], variables locales.
Heap	Memoria dinámica administrada con malloc/free.	NodoLinea y texto de cada línea.
Buffer	Zona temporal para mover bytes.	bloque[4096] recibe datos de read.
fd	Entero que referencia una entrada del kernel.	editor.fd; -1 significa que no hay archivo abierto.
Offset	Posición de la próxima lectura o escritura.	lseek lo mueve al inicio.

```
DISCO -> read(fd, bloque, 4096) -> BUFFER -> malloc -> NODOS DEL HEAP
```

Punto clave
El buffer no guarda permanentemente el documento. Solo transporta un bloque; los nodos conservan las líneas mientras el editor está ejecutándose.

Al salir
close libera el descriptor en el kernel. free libera los nodos en el heap. Son recursos distintos y ambos deben atenderse.

15. Recorridos completos

Lee estas cadenas de izquierda a derecha para explicar el programa oralmente.

Abrir o crear

```
comando o -> editor_abrir -> archivo_abrir -> open
          -> archivo_cargar -> lseek -> read -> realloc
          -> lista_agregar -> malloc -> memcpy -> enlazar
```

Agregar

```
comando a -> editor_agregar -> strlen -> lista_agregar
          -> malloc -> memcpy -> enlazar al final
          -> archivo_agregar_linea -> lseek(SEEK_END) -> write
```

Borrar

```
comando d -> strtol -> editor_borrar -> lista_obtener
          -> reconectar vecinos -> free(texto) -> free(nodo)
          -> archivo_guardar -> lseek -> ftruncate -> write
```

Salir

```
comando q -> editor_cerrar -> close(fd) -> lista_vaciar -> free
```

Compilación y arranque

```
make -> compilar cuatro .c -> enlazar ejecutable editor
./editor [archivo] -> main -> inicializar -> ciclo editor>
```

16. Lo que no es una función

Preguntas cortas que pueden aparecer en la sustentación.

Elemento	Clasificación	Significado
`sizeof`	Operador de C	Calcula el tamaño de un tipo u objeto.
`NULL`	Macro	Representa un puntero nulo.
`STDIN_FILENO`	Macro POSIX	Descriptor 0.
`STDOUT_FILENO`	Macro POSIX	Descriptor 1.
`O_RDWR | O_CREAT`	Flags combinados por bits	Lectura/escritura y creación si falta.
`SEEK_SET`	Constante	Interpreta el offset desde el inicio.
`SEEK_END`	Constante	Interpreta el offset desde el final.
`0644`	Literal octal de modo	rw-r--r-- antes de aplicar umask.
`errno`	Estado de error por hilo	perror consulta su valor actual.

Resumen para memorizar

Grupo	Lista completa
Syscalls (6)	open, read, write, lseek, ftruncate, close
stdio.h (6)	printf, fprintf, fgets, fflush, feof, perror
stdlib.h (4)	malloc, realloc, free, strtol
string.h (6)	strcmp, strncmp, strchr, strcspn, strlen, memcpy

Respuesta corta de sustentación
El editor independiente carga con open/read, representa cada línea como un nodo del heap y persiste con lseek/write/ftruncate. El comando q ejecuta close, libera todos los nodos con free y termina normalmente main.

17. Verificación, entrega y fuentes

Las evidencias se pueden repetir con comandos incluidos en el paquete.

Comprobación	Resultado esperado
`make clean && make`	Un ejecutable editor y cero advertencias del compilador.
`make test`	Mensaje OK para comandos, bordes, persistencia y POSIX.
Archivo sin LF final	a inserta el separador y crea una línea nueva real.
Inicio con argumento	./editor archivo abre o crea desde main.
Limpieza	q llama close y lista_vaciar libera texto y nodo.

Comandos de evidencia

```
cd Entrega_Final_Editor_Independiente_Javier_Sierra
make clean && make
make test
./editor
```

Fuentes técnicas consultadas

- Linux man-pages: open(2), read(2), write(2), lseek(2), ftruncate(2) y close(2): https://man7.org/linux/man-pages/
- Consigna de la Evaluación Práctica del primer parcial, suministrada por la asignatura.
- Indicación directa del docente: entregar el editor como ejecutable independiente, sin integración con el Shell.

Estado de entrega
Código fuente, Makefile, pruebas, manuales, este PDF y guion del video están incluidos. El estudiante únicamente debe grabar la demostración con su voz.